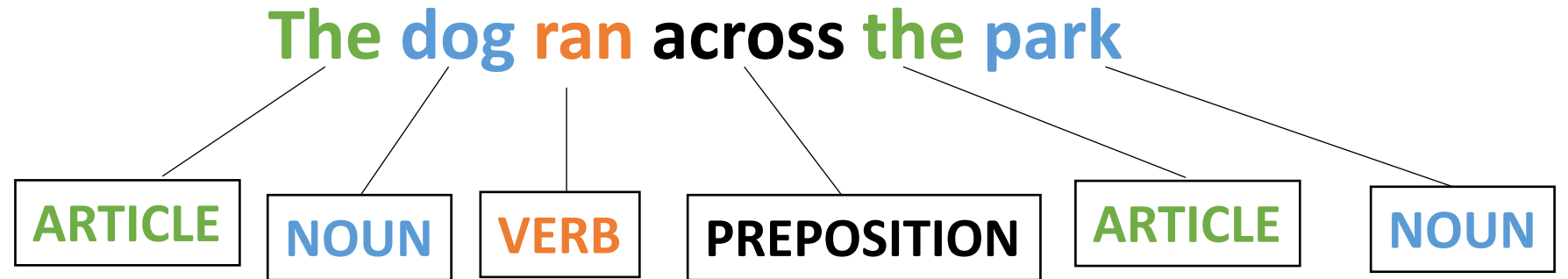


CSE110A: Compilers



- **Topics:**

- *Finishing up Scanners*
 - *Token actions*
 - *PLY Scanner*

Announcements

Office Hours for me:

- *Starting on Thursday, please see calendar.*

Homework Clarification:

- *Starting Module 2: Parsing*
 - *For Part 2*
 - *HNUM: a hex number, like in C, should start with a 0x followed by digits which include a-f (case insensitive)*

Announcements

Homework Clarification:

- *For Part 4: You should not hard code the RedEx, you should generate it given the list of tokens.*
- *How we will grade:*
 - *Your tokens will be graded using our solution scanner importing your tokens.*
 - *We will put in our own tokens to grade your SOS and NG scanners*
 - *You don't need to reorder the tokens, you can assume they are given to you in a good order.*

Finishing up scanner implementations

Scanners we have discussed

- *Naïve Scanner*
- *RE based scanners*
 - Exact match (EM) scanners
 - Start-of-string (SOS) scanners
 - named group (NG) scanners

Which one to use?

Complex decision with performance, expressivity, and token requirements

In practice

- Most scanner generators systems have SOS semantics
 - You can reason about tokens independently
 - Use fast "match" implementations under the hood
- Mainstream compilers:
 - have hand coded and hand optimized scanners
 - Very fast
 - Hard to modify
 - *Only worth it to do this if you have the need and time*

Moving on

- Token actions
 - Replacement
 - Keywords
 - Error reporting
- Scanner error recovery

Moving on

- **Token actions**
 - Replacement
 - Keywords
 - Error reporting
- Scanner error recovery

First class functions

- A programming language is said to have first class functions if functions can be stored as variables
- Python has great support for this
- Functional languages have great support (and compiler helps out by checking types in static languages)
- In C++
 - Classically: function pointers
 - Newer: supports lambdas and improvements
 - (starting on C++11 (2011), C++14, C++17, C++20)

Functions as part of a token definition

- In our scanners, we give them as the 3rd element in the token tuple definition
- A token action takes in a lexeme and returns a lexeme.
 - Possibly the same lexeme

They generally do three things:

- modify a value
- refine a token
- modify the scanner state

Functions as part of a token definition

- Once a token is matched, its token action is called on its lexeme,
- and the lexeme it returns is returned from the scanner,
- **Code example** in the EM (see code in homework 1)

Examples

Token actions generally do three things:

- **modify a value**
- refine a token
- modify the scanner state

Modify a value

- Example using natural language

Modify a value

• PRONOUN	=	{His, Her, Their}
• NOUN	=	{Dog, Cat, Car, Park}
• VERB	=	{Slept, Ate, Ran}
• ADJECTIVE	=	{Purple, Spotted, Old}

Tokens

Tokens Definitions

Modify a value

• PRONOUN	=	{His, Her, Their}
• NOUN	=	{Dog, Cat, Car, Park}
• VERB	=	{Slept, Ate, Ran}
• ADJECTIVE	=	{Purple, Spotted, Old}

Tokens

Tokens Definitions

Example:
Can change any pronoun value
to gender neutral ("Their")

(demo3b)

Modify a Value

Example using types:

- Some ML frameworks experiment with lower precision, e.g., **float16**
- Change code to use lower precision

```
float  x,  y;  
return x+y;
```

*Scanner can easily
change float16 to
float with a token
action*

```
float16 x,  y;  
return x+y;
```


Examples

Token actions generally do three things:

- modify a value
- **refine a token**
- modify the scanner state

Keywords: Reserved Words

TOKENS

ID = [a-z] +

NUM = [0-9] +

ASSIGN = "="

PLUS = "+"

MULT = "*"

IGNORE = [" ", "\n"]

KEYWORDS

[(INT, "int"), (FLOAT, "float") ...]

Keywords: Reserved Words

TOKENS

INT = "int"

ID = [a-z] +

NUM = [0-9] +

ASSIGN = "="

PLUS = "+"

MULT = "*"

IGNORE = [" ", "\n"]

KEYWORDS

[(INT, "int"), (FLOAT, "float") ...]

**WHAT ABOUT
THIS APPROACH?**

**Note: Modern C has
about 44 keywords,
Modern C++ about
90 keywords.**

Keywords: Reserved Words

TOKENS

```
ID      = [a-z] +  
NUM     = [0-9] +  
ASSIGN  = "="  
PLUS    = "+"  
MULT    = "*"  
IGNORE  = [" ", "\n"]
```

KEYWORDS

```
[ (INT, "int"), (FLOAT, "float") ... ]
```

*Code example using
EM Scanner*

(demo3c)

Examples

Token actions generally do three things:

- modify a value
- refine a token
- **modify the scanner state**

Modifying State

In the common case, we can create a scanner and then update a class member in a token action

EM Scanner example:

Advanced Topic

- Recovering from errors (syntax highlighting)
 - Show clang example:
 - use the command line option: `-fsyntax-only -Xclang -dump-tokens`
 - try to tokenize weird symbols, such as ```
- return an error token and try to recover
 - eating one character
 - eating until a space
 - eating until a newline

Next Topics

- Using a scanner generator:
 - They have their own designs and it is important to understand trade-offs and design decisions
- Classically:
 - Lex and Flex
- Modern:
 - Antlr (ANother Tool for Language Recognition)
- A good in-between:
 - PLY - a Lex and Yacc implementation in Python

Lex/Flex

- Old tools - input is a token specification file.
Produces a complicated C file that you would include in your project
- New language technology makes things a lot easier
(higher order functions, fast RE matchers, etc.)

PLY

- written mostly for education purposes. Uses only core python features
- Personally, I have used it many times for little compiler projects
- Documented to be a python implementation of Lex, but uses a much nicer interface

How to use PLY's Scanner

Scanner Demo

- *Library import*

```
import ply.lex as lex
```

- *Token list*

```
tokens = ["ADJECTIVE", "NOUN", "VERB", "ARTICLE"]
```

- *Token specification*

```
t_ADJECTIVE = "old|purple|spotted"
```

```
t_NOUN = "dog|computer|car"
```

```
t_ARTICLE = "the|my|a|your"
```

```
t_VERB = "ran|crashed|accelerated"
```

Scanner Demo

- *Build the lexer*

`lexer = lex.lex()` *what happens?*

- *Need an error function*

Error handling rule

```
def t_error(t):
```

```
    print("Illegal character '%s'" %  
t.value[0])
```

```
    exit(1)
```

Scanner Demo

- *Now give the lexer some input*

```
lexer.input("dog")
```

- *The lexer streams the input, we need to stream the tokens:*

```
# Tokenize
```

```
while True:
```

```
    tok = lexer.token()
```

```
    if not tok:
```

```
        break    # No more input
```

```
    print(tok)
```

Scanner Demo

- *output:*

line number (1 indexed)

LexToken (NOUN, 'dog', 1, 0)

number of characters streamed (0 indexed)

- *try a longer string:*

```
lexer.input("dog computer")
```

What happens?

Scanner Demo

- *Need to add a token for whitespace!*

```
tokens = ["ADJECTIVE", "NOUN", "VERB", "ARTICLE",  
"...WHITESPACE"]
```

```
t_WHITESPACE = '\ '
```

- *Now we can lex:*

```
LexToken(NOUN,'dog',1,0)
```

```
LexToken(WHITESPACE,' ',1,3)
```

```
LexToken(NOUN,'computer',1,4)
```


Scanner Demo

- *Now we can do a sentence*

```
lexer.input("my spotted dog ran")
```

```
LexToken(ARTICLE,'my',1,0)
```

```
LexToken(WHITESPACE,' ',1,2)
```

```
LexToken(ADJECTIVE,'spotted',1,3)
```

```
LexToken(WHITESPACE,' ',1,10)
```

```
LexToken(NOUN,'dog',1,11)
```

```
LexToken(WHITESPACE,' ',1,14)
```

```
LexToken(VERB,'ran',1,15)
```

Can we clean this up?

Scanner Demo

- *We can ignore whitespace*

```
#t_WHITESPACE = '\
```

```
t_ignore = ' '
```

gets simplified to:

```
LexToken(ARTICLE,'my',1,0)  
LexToken(WHITESPACE,' ',1,2)  
LexToken(ADJECTIVE,'spotted',1,3)  
LexToken(WHITESPACE,' ',1,10)  
LexToken(NOUN,'dog',1,11)  
LexToken(WHITESPACE,' ',1,14)  
LexToken(VERB,'ran',1,15)
```

```
LexToken(ARTICLE,'my',1,0)  
LexToken(ADJECTIVE,'spotted',1,  
3)  
LexToken(NOUN,'dog',1,11)  
LexToken(VERB,'ran',1,15)
```

Scanner Demo

- *What about newlines?*

```
lexer.input("""  
my spotted dog ran  
the old computer crashed  
""")
```

- *Need to add a newline token!*

Scanner Demo

- *What about newlines?*

```
lexer.input("""  
my spotted dog ran  
the old computer crashed  
""")
```

- *Need to add a newline token!*

```
tokens = ["ADJECTIVE", "NOUN", "VERB", "ARTICLE", "NEWLINE"]  
t_NEWLINE = "\\n"
```

Scanner Demo

```
LexToken(NEWLINE,'\n',1,0)  
LexToken(ARTICLE,'my',1,1)  
LexToken(ADJECTIVE,'spotted',1,4)  
LexToken(NOUN,'dog',1,12)  
LexToken(VERB,'ran',1,16)  
LexToken(NEWLINE,'\n',1,19)  
LexToken(ARTICLE,'the',1,20)
```

Line numbers are not updating

Scanner Demo

- *Token actions*

```
t_NEWLINE = "\\n"
```

Changes into:

```
def t_NEWLINE(t):  
    "\\n"  
    t.lexer.lineno += 1  
    return t
```

docstring is the regex, lexer object which has a lineno attribute.

If we don't return anything, then it is ignored.

Scanner Demo

- *Example: changing gendered pronouns into gender neutral pronouns*

```
tokens = ["ADJECTIVE", "NOUN", "VERB", "ARTICLE", "NEWLINE",  
"PRONOUN"]
```

```
t_PRONOUN = "her|his|their"
```

```
lexer.input("""  
his spotted dog ran  
her old computer crashed  
""")
```

Scanner Demo

- *Add a token action:*

```
def t_PRONOUN(t):  
    "her|his|their"  
    if t.value in ["his", "her"]:  
        t.value = "their"  
    return t
```

Now output will have all gender neutral pronouns!